



VIT[®]

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

Fall Semester 2025-26

Lab Assignment – 5

Slot: L13+L14

Class: VL2025260105679

Branch: B.tech CSBS

Course code & title: CBS3005

Cloud, Microservices and Applications

Faculty name: Nithya K

DA by:

Kartikey Gupta

22BBS0105

Task 2: Analyze API Activities using AWS CloudTrail

Description:

Enable AWS CloudTrail to monitor and log all API activities in the AWS account. Also, analyze events to detect unauthorized or unusual access.

Steps:

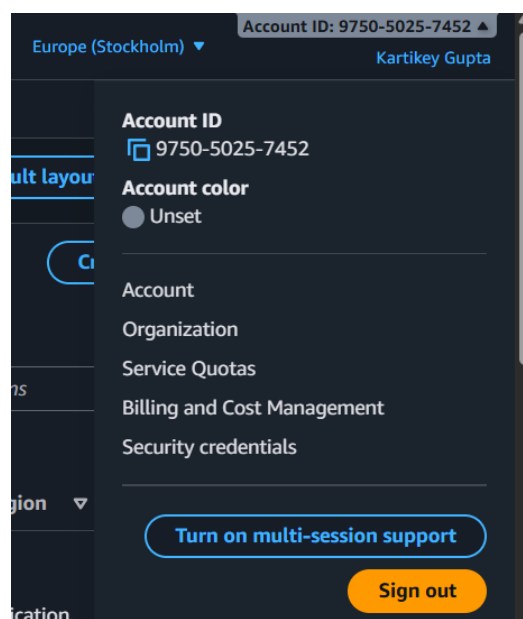
1. Go to CloudTrail Console → Trails → Create Trail.
2. Choose:
 - Trail name: MySecurityTrail
 - Apply trail to All regions
 - Store logs in a new S3 bucket
 - Enable CloudWatch Logs Integration (optional).
3. Perform a few activities in your AWS account:
 - Launch or stop an EC2 instance.
 - Create an S3 bucket.
4. Return to CloudTrail → Event History and view recent events.
5. Identify:
 - Who performed the action (IAM user or role)
 - Time of action
 - Source IP address
 - Affected resource

Expected Output:

- Screenshot of CloudTrail event list.
- Sample JSON event record for one operation.
- Short explanation of how CloudTrail supports auditing and incident investigation.

Step 1: Log in to the AWS Management Console

- Sign in to your AWS account with **Administrator access**.
- In the search bar, type “**CloudTrail**” and open the **CloudTrail** service.



Step 2: Create a New Trail

1. In the **CloudTrail Console**, go to **Trails** → click **Create trail**.
2. Enter details:
 - **Trail name:** MySecurityTrail
 - **Apply trail to all regions:** Yes
 - **Storage location:**
 - Choose **Create a new S3 bucket**.
 - Enter a unique bucket name (mysecuritytrail-logs-22bbs0105).

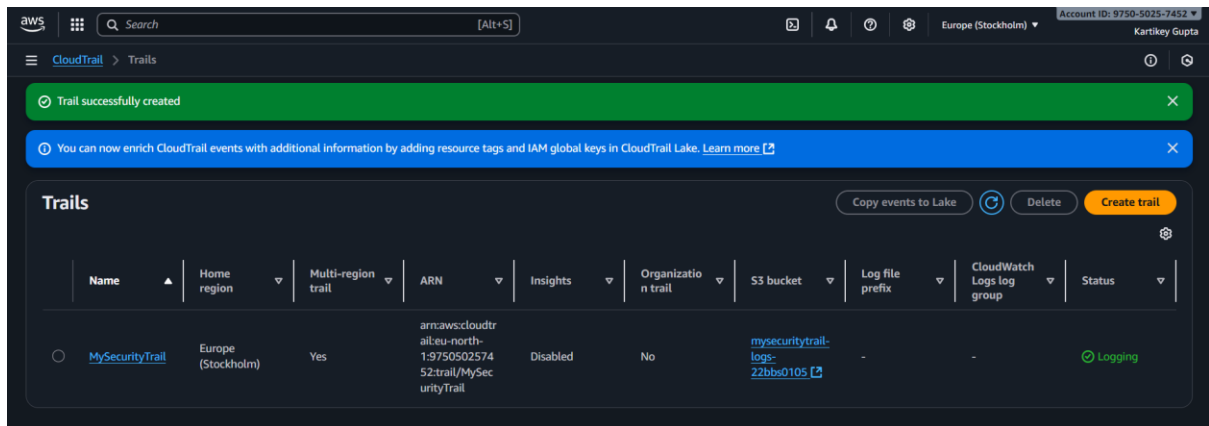
The screenshot shows the 'Choose trail attributes' page in the AWS CloudTrail console. The left sidebar indicates the current step is 'Choose trail attributes'. The main content area is titled 'Choose trail attributes' and contains the following sections:

- General details**: A trail created in the console is a multi-region trail. [Learn more](#)
- Trail name**: Enter a display name for your trail. The input field contains 'MySecurityTrail'. Below the field, it states: '3-128 characters. Only letters, numbers, periods, underscores, and dashes are allowed.'
- Storage location**: Two radio buttons are present. 'Create new S3 bucket' is selected, with the subtext 'Create a bucket to store logs for the trail.' The other option is 'Use existing S3 bucket' with the subtext 'Choose an existing bucket to store logs for this trail.'
- Trail log bucket and folder**: Enter a new S3 bucket name and folder (prefix) to store your logs. Bucket names must be globally unique. The input field contains 'mysecuritytrail-logs-22bbs0105'. Below the field, it states: 'Logs will be stored in mysecuritytrail-logs-22bbs0105/AWSLogs/975050257452'.
- Log file SSE-KMS encryption**: A checkbox labeled 'Enabled' is checked.

3. Click **Next**, review the settings, and select **Create trail**.

The screenshot shows the 'Choose log events' page in the AWS CloudTrail console. The left sidebar indicates the current step is 'Choose log events'. The main content area is titled 'Choose log events' and contains the following sections:

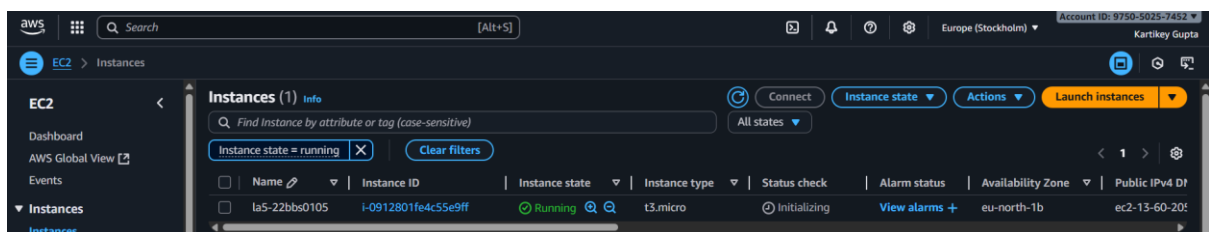
- Events**: Record API activity for individual resources, or for all current and future resources in AWS account. [Additional charges apply](#)
- Event type**: Choose the type of events that you want to log. Three checkboxes are present: 'Management events' (checked), 'Data events' (unchecked), and 'Insights events' (unchecked). Below these, there is a section for 'Network activity events' which is currently unchecked.
- Management events**: Management events show information about management operations performed on resources in your AWS account. A message box states: 'No additional charges apply to log management events on this trail because this is your first copy of management events.'



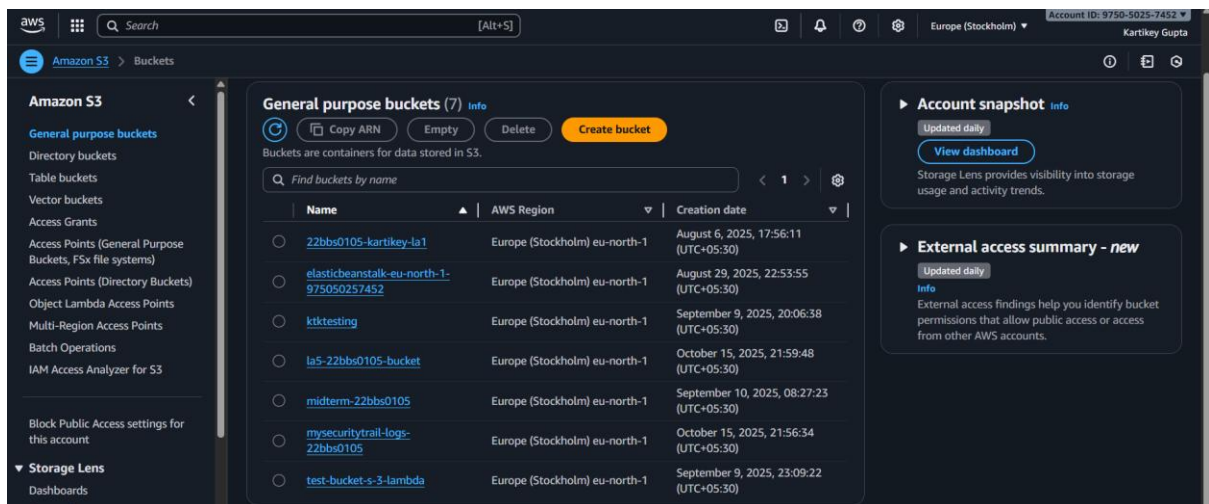
Step 3: Perform AWS Activities to Generate Logs

Perform a few actions to generate CloudTrail events:

- Go to the **EC2 Console** → **Launch Instance**



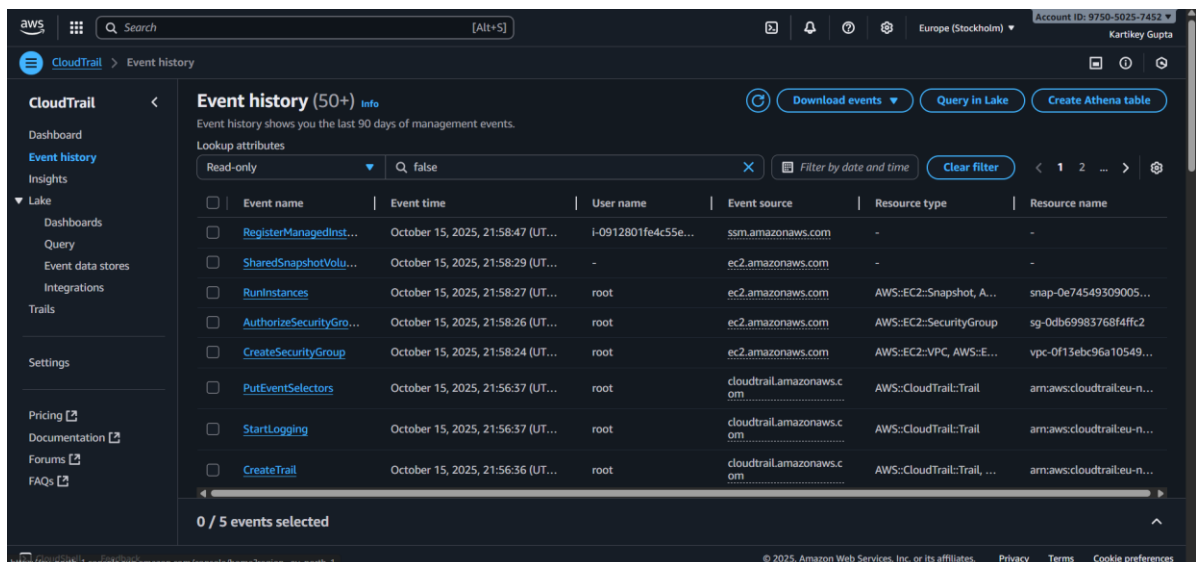
- Go to the **S3 Console** → **Create a new S3 bucket**.



Wait 2–5 minutes after performing these actions — CloudTrail may take some time to log them.

Step 4: View CloudTrail Events

- Return to the **CloudTrail Console** → Click **Event History** (left menu).
- You'll see a list of recorded events.



3. Use filters to narrow down results:

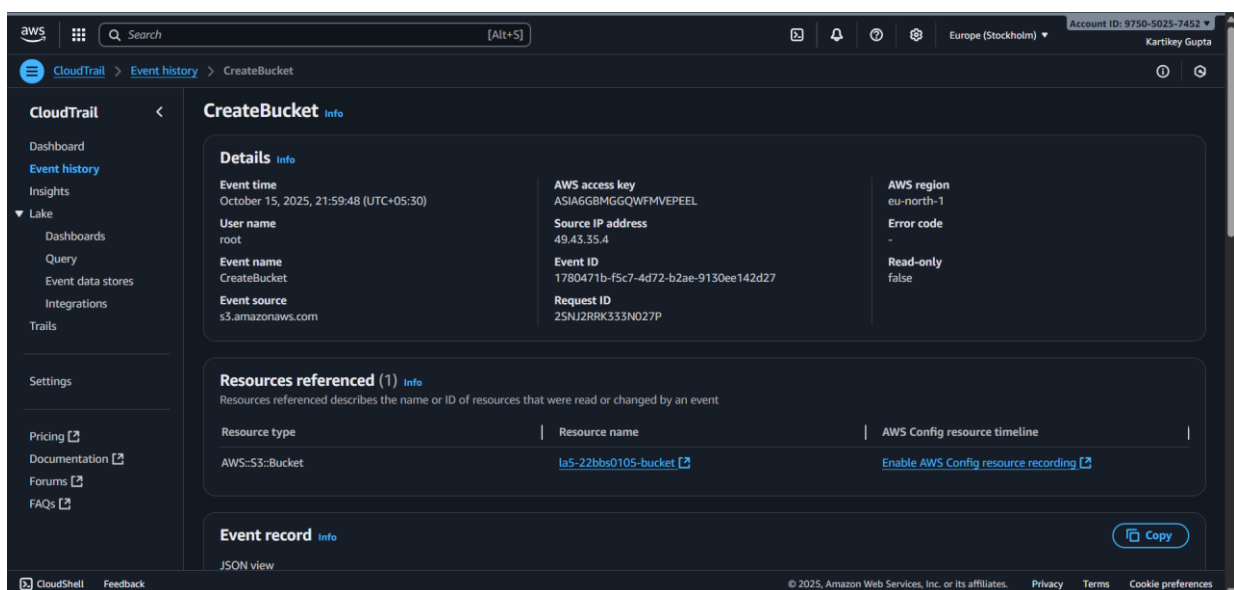
- **Event Name:** RunInstances, StopInstances, CreateBucket, etc.
- **Event Source:** e.g., ec2.amazonaws.com, s3.amazonaws.com.

Step 5: Identify Key Details

Click on any event to open its details.

From the JSON record, note the following:

- **Who performed the action:** (Look for "userIdentity": {"userName": ...})
- **Time of action:** "eventTime"
- **Source IP address:** "sourceIPAddress"
- **Affected resource:** "resources" or "requestParameters"



Step 6: Export Required Output

- Take a screenshot of your **CloudTrail Event History** page showing recorded activities.
- Open one event → click **View Event** → copy the JSON content.

```
{
  "eventVersion": "1.11",
  "userIdentity": {
    "type": "Root",
    "principalId": "975050257452",
    "arn": "arn:aws:iam::975050257452:root",
    "accountId": "975050257452",
    "accessKeyId": "ASIA6GBMGGQWFMVEPEEL",
    "sessionContext": {
      "attributes": {
        "creationDate": "2025-10-15T16:18:12Z",
        "mfaAuthenticated": "true"
      }
    }
  },
  "eventTime": "2025-10-15T16:29:48Z",
  "eventSource": "s3.amazonaws.com",
  "eventName": "CreateBucket",
  "awsRegion": "eu-north-1",
  "sourceIPAddress": "49.43.35.4",
  "userAgent": "[Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/140.0.0.0 Safari/537.36]",
  "requestParameters": {
    "CreateBucketConfiguration": {
      "LocationConstraint": "eu-north-1",
      "xmlns": "http://s3.amazonaws.com/doc/2006-03-01/"
    }
  },
  "bucketName": "la5-22bbs0105-bucket",
```

```
    "Host": "la5-22bbs0105-bucket.s3.eu-north-1.amazonaws.com"
  },
  "responseElements": null,
  "additionalEventData": {
    "SignatureVersion": "SigV4",
    "CipherSuite": "TLS_CHACHA20_POLY1305_SHA256",
    "bytesTransferredIn": 192,
    "AuthenticationMethod": "AuthHeader",
    "x-amz-id-2":
"Aj6WmmJvdKYIEw8DXu99620jKEkbadNrZB03+piEk+GpknkdSdPletsj+B4oOnTnoiyrwwqWq8k=",
    "bytesTransferredOut": 0
  },
  "requestID": "2SNJ2RRK333N027P",
  "eventID": "1780471b-f5c7-4d72-b2ae-9130ee142d27",
  "readOnly": false,
  "eventType": "AwsApiCall",
  "managementEvent": true,
  "recipientAccountId": "975050257452",
  "eventCategory": "Management",
  "tlsDetails": {
    "tlsVersion": "TLSv1.3",
    "cipherSuite": "TLS_CHACHA20_POLY1305_SHA256",
    "clientProvidedHostHeader": "la5-22bbs0105-bucket.s3.eu-north-1.amazonaws.com"
  }
}
```

Detail	Value
Who performed the action (IAM user or role)	Root user (arn:aws:iam::975050257452:root)
Time of action	2025-10-15T16:29:48Z
Source IP address	49.43.35.4
Affected resource	S3 bucket: la5-22bbs0105-bucket

Similarly, we will have log information for ec2 instance.

Task 2: Configure CloudWatch for EC2 Instance Monitoring

Description:

Create and monitor a simple EC2 instance using **Amazon CloudWatch**, configure alarms, monitor system metrics, and analyze performance data to understand how CloudWatch helps in proactive threat and performance monitoring.

Steps:

1. Launch a **t2.micro EC2 instance** (Free Tier eligible) running Amazon Linux 2.
2. Install and configure the **CloudWatch Agent** using:

```
sudo yum install amazon-cloudwatch-agent -y
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-config-wizard
```

3. Choose to monitor:
 - o CPU utilization
 - o Memory usage
 - o Disk I/O

4. Start the CloudWatch agent:

```
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl \
-a fetch-config -m ec2 -c file:/opt/aws/amazon-cloudwatch-agent/bin/config.json -s
```

5. Go to **CloudWatch Console** → **Metrics** → **EC2** and verify data collection.

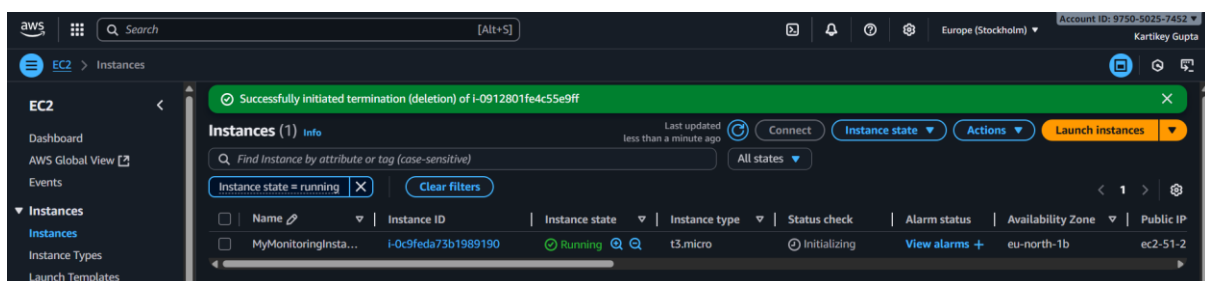
6. Create a **CloudWatch Alarm**:
 - o Metric: CPUUtilization
 - o Condition: >70% for 2 consecutive periods
 - o Notification: via **SNS topic** (send email alert).

Expected Output:

- CloudWatch dashboard showing metrics (CPU, Memory).
- Alarm configuration screenshot.
- Email alert showing triggered event.

Step 1: Launch EC2 Instance

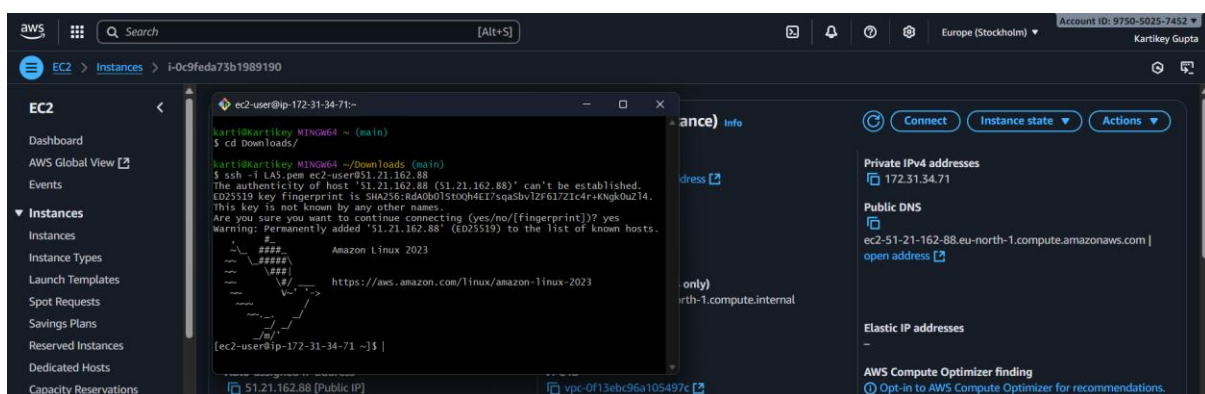
1. Go to the **AWS Management Console** → Search for **EC2** → Click **Instances** → **Launch instance**.
2. Configuration details:
 - **Name:** MyMonitoringInstance
 - **AMI:** Amazon Linux 2 (Free Tier eligible)
 - **Instance type:** t2.micro
 - **Key pair:** Select existing or create a new one.
 - **Security group:** Allow SSH (port 22).
3. Click **Launch Instance** and wait until the instance state becomes **running**.



Step 2: Connect to the Instance

1. Select your instance → Click **Connect** → Choose **EC2 Instance Connect** or **SSH**.
Example (for SSH):

`ssh -i your-key.pem ec2-user@<public-ip>`



Step 3: Install CloudWatch Agent

Run the following commands inside your EC2 instance terminal:

`sudo yum install amazon-cloudwatch-agent -y`

```
[ec2-user@ip-172-31-34-71 ~]$ sudo yum install amazon-cloudwatch-agent -y
Amazon Linux 2023 Kernel Livepatch repository 221 kB/s | 26 kB 00:00
Dependencies resolved.
=====
Package Arch Version Repository Size
=====
Installing:
amazon-cloudwatch-agent x86_64 1.300057.2-1.amzn2023 amazonlinux 135 M
Transaction Summary
=====
Install 1 Package

Total download size: 135 M
Installed size: 471 M
Downloading Packages:
amazon-cloudwatch-agent-1.300057.2-1.amzn2023.x 69 MB/s | 135 MB 00:01
-----
Total 68 MB/s | 135 MB 00:02
Running transaction check
Transaction check succeeded.
Running transaction test
```

Step 4: Configure CloudWatch Agent

Start the interactive setup wizard:

```
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-config-wizard
```

When prompted:

- Choose to monitor **EC2 instance metrics**.
- Select:
 - **CPU utilization**
 - **Memory usage**
 - **Disk I/O**
- Choose to **store the config file locally** (default path: `/opt/aws/amazon-cloudwatch-agent/bin/config.json`).

```

    "mem": {
      "measurement": [
        "mem_used_percent"
      ],
      "metrics_collection_interval": 60
    }
  }
}

Please check the above content of the config.
The config file is also located at /opt/aws/amazon-cloudwatch-agent/bin/config.json.
Edit it manually if needed.
Do you want to store the config in the SSM parameter store?
1. yes
2. no
Default choice: [1]:
Program exits now.
[ec2-user@ip-172-31-34-71 ~]$
```

Step 5: Start CloudWatch Agent

Start the agent using the configuration file:

```
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl \
-a fetch-config -m ec2 -c file:/opt/aws/amazon-cloudwatch-agent/bin/config.json -s
```

Confirm the agent is running:

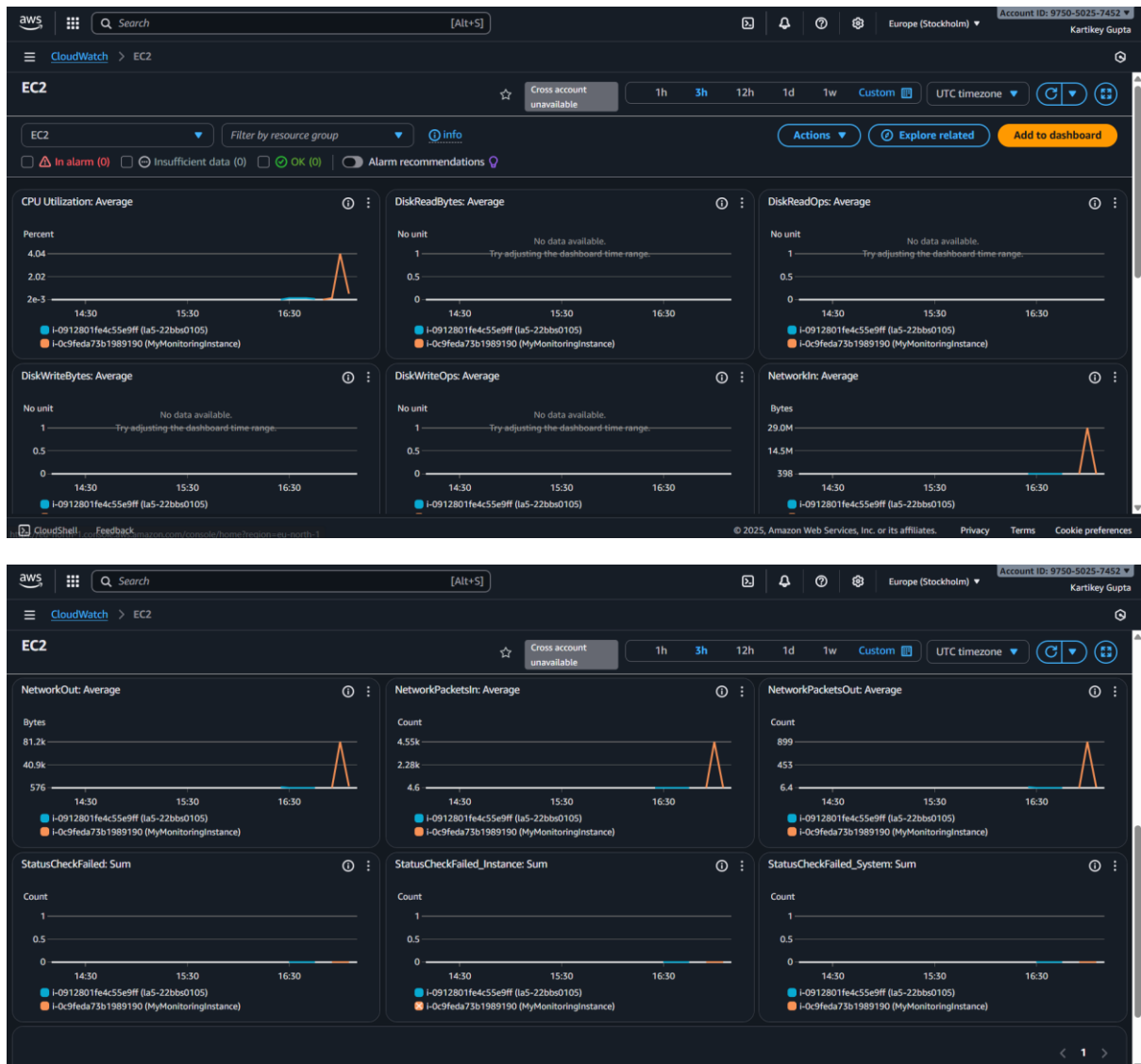
```
sudo systemctl status amazon-cloudwatch-agent
```

```
ec2-user@ip-172-31-34-71 ~]$ sudo systemctl status amazon-cloudwatch-agent
● amazon-cloudwatch-agent.service - Amazon CloudWatch Agent
   Loaded: loaded (/etc/systemd/system/amazon-cloudwatch-agent.service; enabled; vendor preset: enabled)
   Active: active (running) since Wed 2025-10-15 17:08:28 UTC; 10s ago
     Main PID: 2497 (amazon-cloudwat)
       Tasks: 7 (limit: 1053)
      Memory: 49.9M
         CPU: 313ms
    CGroup: /system.slice/amazon-cloudwatch-agent.service
            └─2497 /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent

Oct 15 17:08:28 ip-172-31-34-71.eu-north-1.compute.internal start-amazon-cloudw>
Oct 15 17:08:28 ip-172-31-34-71.eu-north-1.compute.internal start-amazon-cloudw>
Oct 15 17:08:28 ip-172-31-34-71.eu-north-1.compute.internal start-amazon-cloudw>
Oct 15 17:08:28 ip-172-31-34-71.eu-north-1.compute.internal start-amazon-cloudw>
Oct 15 17:08:28 ip-172-31-34-71.eu-north-1.compute.internal start-amazon-cloudw>
Oct 15 17:08:28 ip-172-31-34-71.eu-north-1.compute.internal start-amazon-cloudw>
Oct 15 17:08:28 ip-172-31-34-71.eu-north-1.compute.internal start-amazon-cloudw>
Oct 15 17:08:28 ip-172-31-34-71.eu-north-1.compute.internal start-amazon-cloudw>
Oct 15 17:08:28 ip-172-31-34-71.eu-north-1.compute.internal start-amazon-cloudw>
Oct 15 17:08:28 ip-172-31-34-71.eu-north-1.compute.internal start-amazon-cloudw>
lines 1-20/20 (END)
```

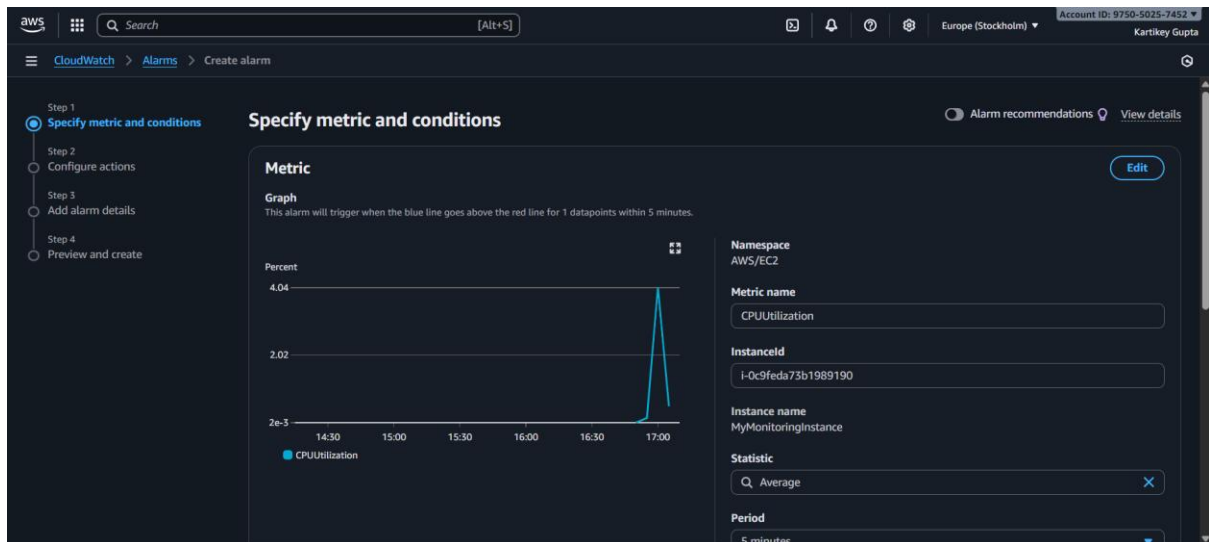
Step 6: Verify Metrics in CloudWatch

1. Go to the **CloudWatch Console** → **Metrics** → **All metrics** → **EC2**.
2. Check for metrics such as:
 - CPUUtilization
 - mem_used_percent
 - disk_write_bytes / disk_read_bytes
3. You should see your instance ID listed; click it to view real-time graphs.



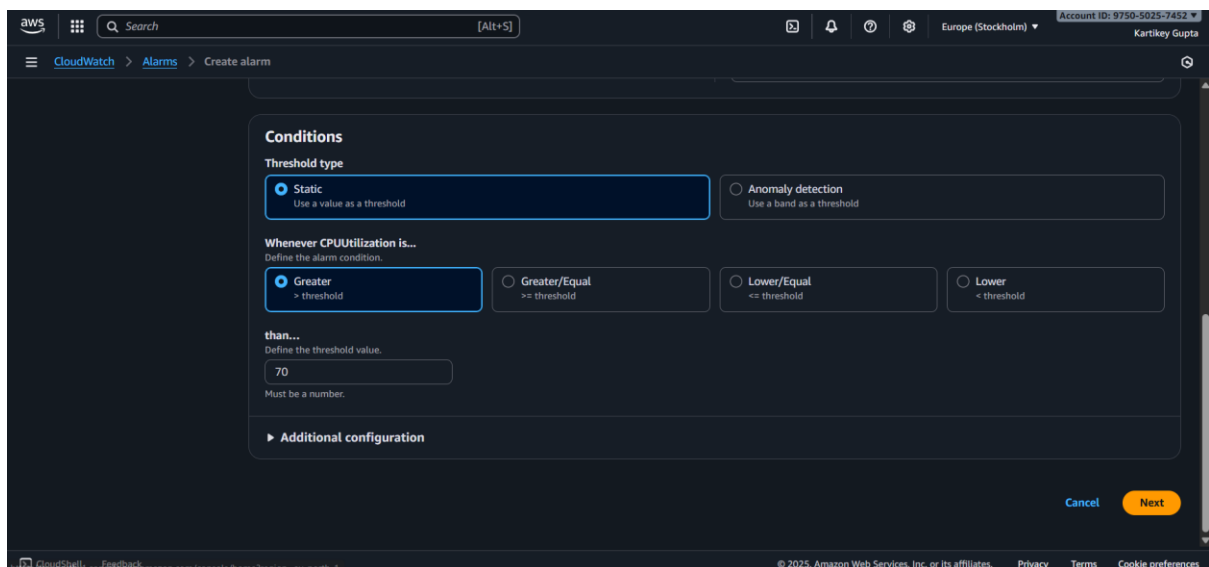
Step 7: Create CloudWatch Alarm

1. In the **CloudWatch Console**, go to **Alarms** → **All alarms** → **Create alarm**.
2. Choose metric:
 - **Browse metrics** → **EC2** → **Per-Instance Metrics** → **CPUUtilization**.
 - Select your instance → Click **Select metric**.



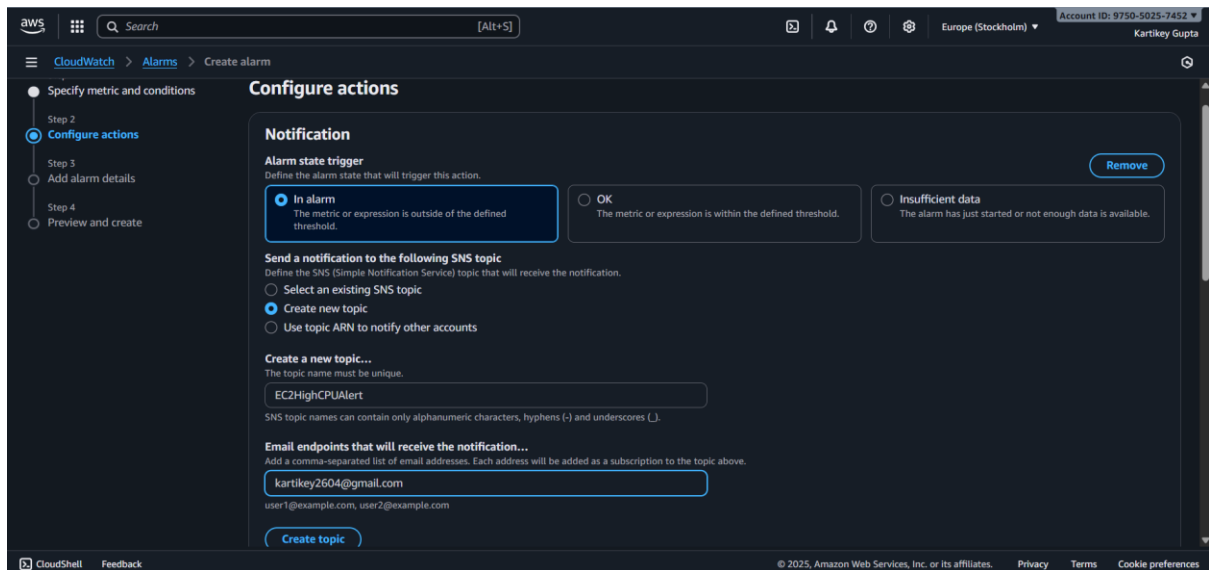
3. Set condition:

- **Threshold type:** Static
- **Whenever CPUUtilization > 70**
- **For 2 consecutive 5-minute periods**

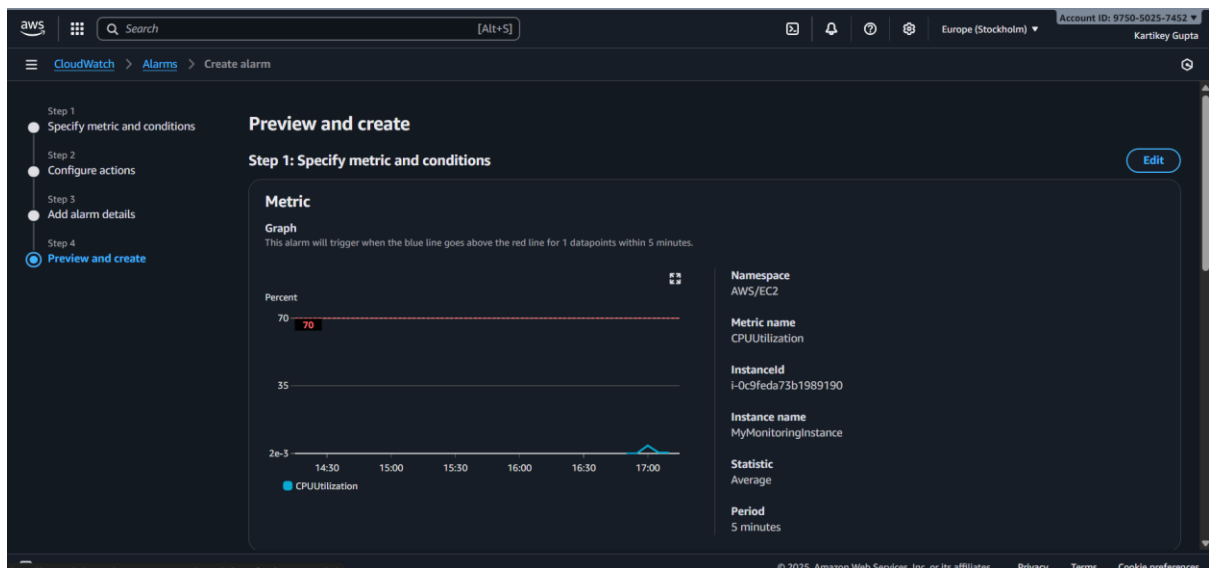


4. Create **SNS topic** for email alerts:

- Click **Create new topic**
- Enter topic name: EC2HighCPUALert
- Add your **email address** for notifications.
- Confirm subscription from the email you receive.



5. Click Next → Create Alarm.



Step 8: Trigger Alarm

To test:

```
sudo stress --cpu 2 --timeout 120
```

(Install stress tool if needed: `sudo yum install stress -y`)

After a few minutes, CPU utilization should exceed 70%, and CloudWatch will trigger the alarm and send an email notification.

```
Installed:
 stress-1.0.7-2.amzn2023.0.1.x86_64

Complete!
[ec2-user@ip-172-31-34-71 ~]$ sudo stress --cpu 2 --timeout 120
stress: info: [21904] dispatching hogs: 2 cpu, 0 io, 0 vm, 0 hdd
stress: info: [21904] successful run completed in 120s
```

